

Stroke Triage with Multi-Method Explainability

1. Background and Context

Hospitals increasingly rely on predictive analytics systems to assist clinical decision-making in emergency departments. One such application involves triage prioritisation for patients who may be at elevated risk of stroke. In these settings, artificial intelligence systems estimate the probability of high-risk cases so that clinicians can allocate attention more effectively. However, predictive performance alone is not sufficient in medical contexts. Any deployed system must also provide clear, reliable, and defensible explanations for its outputs.

You are part of the Clinical AI Governance Team responsible for developing and evaluating a predictive model named StrokeTriageNet. The system estimates the probability that an incoming emergency patient belongs to a high stroke-risk triage category. Before deployment approval, the hospital board requires both rigorous model validation and a structured explainability comparison across multiple explanation paradigms.

You are required to:

- Build a neural network that predicts stroke triage risk.
- Generate explanations for selected predictions.
- Compare three explanation techniques.
- Recommend which method should be deployed in practice.

2. Dataset Description

You are provided with a dataset titled `stroke_triage_stream.csv` containing 2,000 patient records. The dataset simulates a real-time streaming triage environment using twenty time-based batches. Each record contains ten input features describing patient health indicators including age, blood pressure measurements, glucose level, cholesterol level, heart rate, oxygen saturation, BMI, smoker status, and history of diabetes. The target variable is `stroke_risk_label`, where 1 represents high triage risk and 0 represents lower risk. The column `batch_id` indicates simulated time-based arrival and must not be used as a predictive feature.

You are required to:

- Use the 10 health-related features as model inputs.
- Use `stroke_risk_label` as the target variable.
- Exclude `batch_id` from training.
- Clearly describe how you split the data.

R&xAI Assignment# 1

3. Phase 1: Model Training

In this phase, you must design and train a fully connected neural network using PyTorch. You must split the dataset into training, validation, and test sets using a 60,20,20 split. Proper shuffling must be applied to avoid ordering bias. You must implement a feedforward neural network with ten input features and one probabilistic output. The network must contain at least one hidden layer with a minimum of sixteen neurons. The final output must use a sigmoid activation function. The model must be trained using binary cross-entropy loss. You must report test-set performance using Accuracy, Precision, Recall, and ROC-AUC. You must include training vs validation loss plot.

You must:

- Implement a neural network in PyTorch.
- Train it properly and report performance metrics.
- Save model weights.
- Select 10 test instances for explanation.

4. Phase 2: Explainability Implementation and Evaluation

LIME

You must implement a tabular version of LIME including discretisation of numerical features, generation of perturbed samples, proximity kernel weighting, and fitting a weighted linear surrogate model.

- Implement LIME from scratch.
- Produce per-feature attribution values.
- Explain all 10 selected patients.

SHAP

You must implement Shapley-value-based explanations using either exact Shapley computation or Kernel SHAP. The implementation must satisfy the additivity conditions.

- Compute Shapley values correctly.
- Verify the additivity property.
- Produce per-feature Shapley values for 10 patients.

Integrated Gradients

You must implement Integrated Gradients for your trained neural network including baseline definition, interpolation, gradient accumulation, and validation of the Sensitivity axiom.

- Implement IG using PyTorch gradients.
- Validate the Sensitivity condition.
- Produce per-feature IG attributions for 10 patients.

R&xAI Assignment# 1

5. Comparative Evaluation

You must compare LIME, SHAP, and Integrated Gradients in terms of attribution ranking consistency, computational cost, and theoretical differences between surrogate-based, game-theoretic, and gradient-based approaches. You must conclude with a justified recommendation for clinical deployment.

You must:

- Compare ranking consistency of top features.
- Compare computational cost.
- Discuss theoretical differences.
- Recommend one method with justification.

6. Submission Requirements

You must submit one ZIP file containing all required materials. The submission must include source code for training and explanation implementation, the saved trained model, and three JSON files named `lime_results.json`, `shap_results.json`, and `ig_results.json`. Each JSON file must contain the patient index, predicted risk score, and full attribution vector for each of the ten selected patients.

Your submission must contain:

- **Complete source code.**
- **Saved trained model.**
- **Three explanation JSON files (10 patients each).**